



抖音功耗优化实践

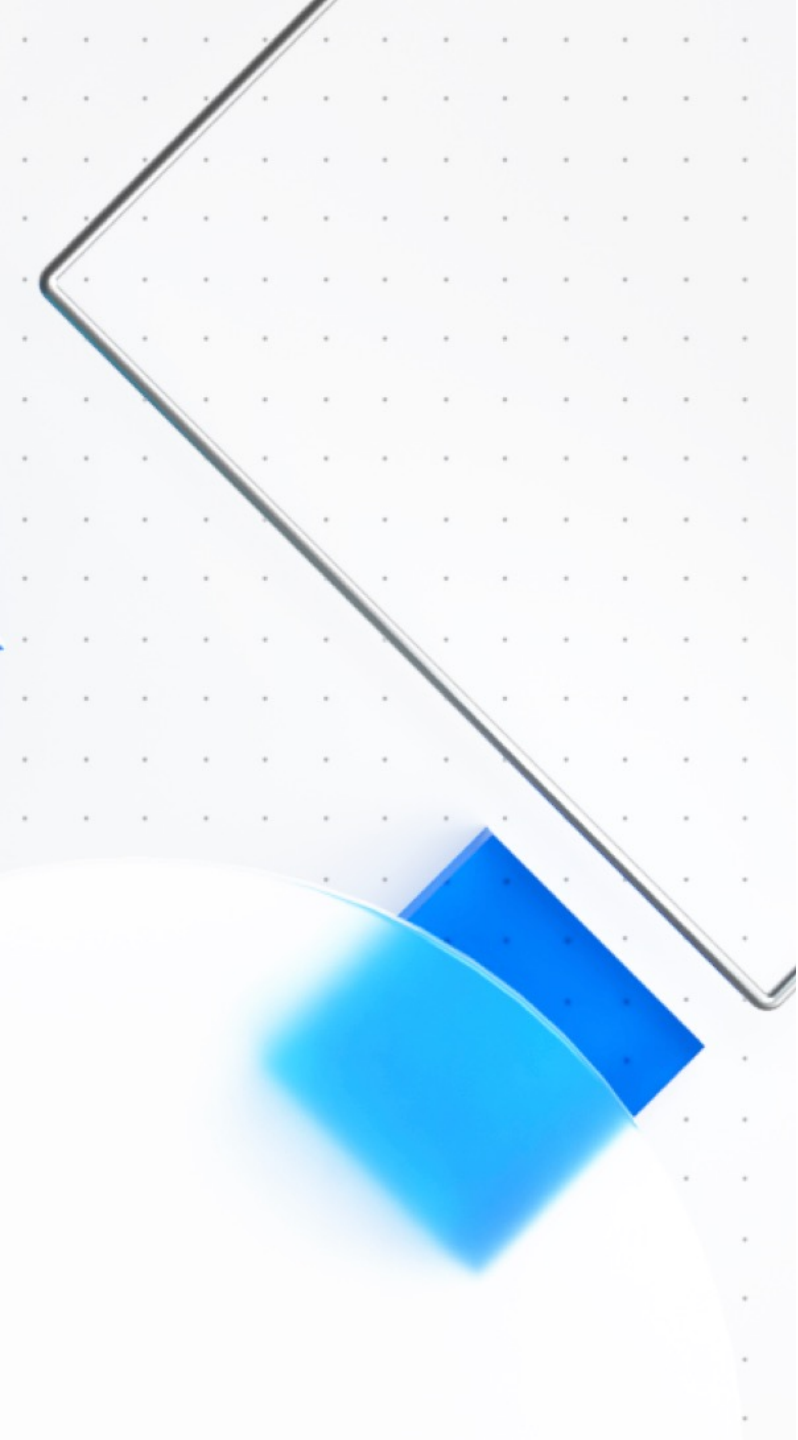
高原

讲师自我介绍



高原

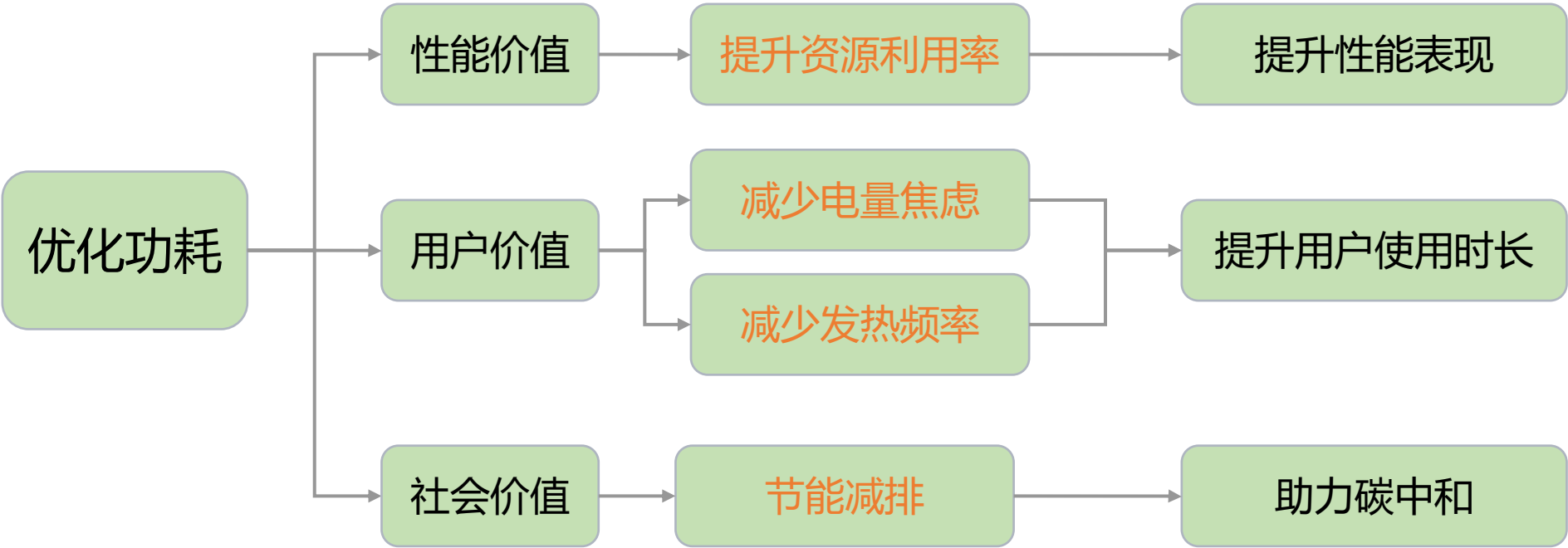
毕业于北京邮电大学，毕业后进入华为从事Android手机系统的功耗优化的工作，2021年加入字节跳动，目前主要负责抖音的功耗体验优化工作，对Android功耗优化有较为丰富的经验

- 
- 01 抖音功耗优化的开展思路
 - 02 优化案例解析
 - 03 总结&展望

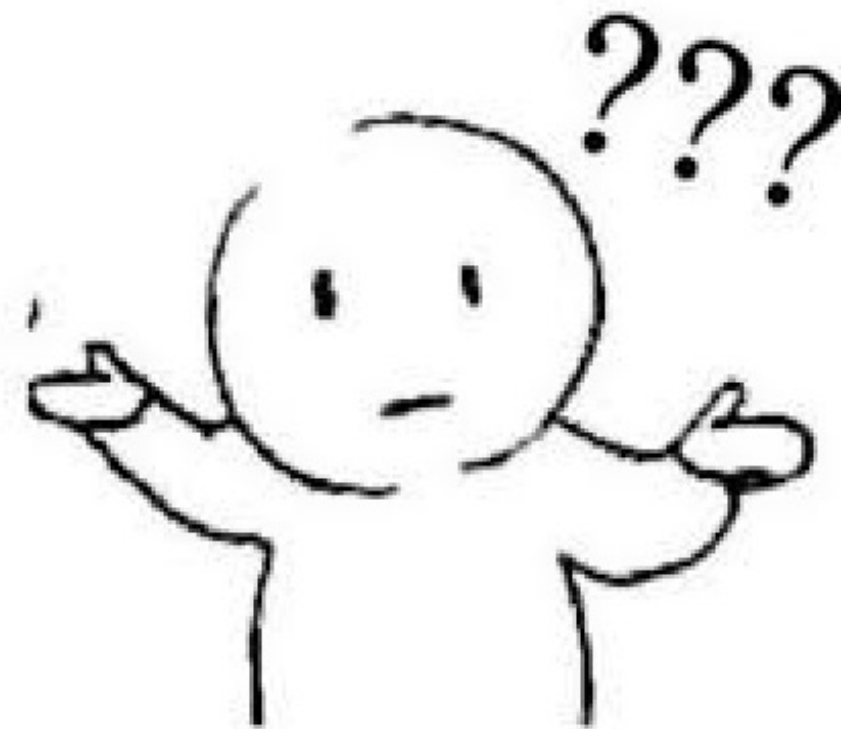
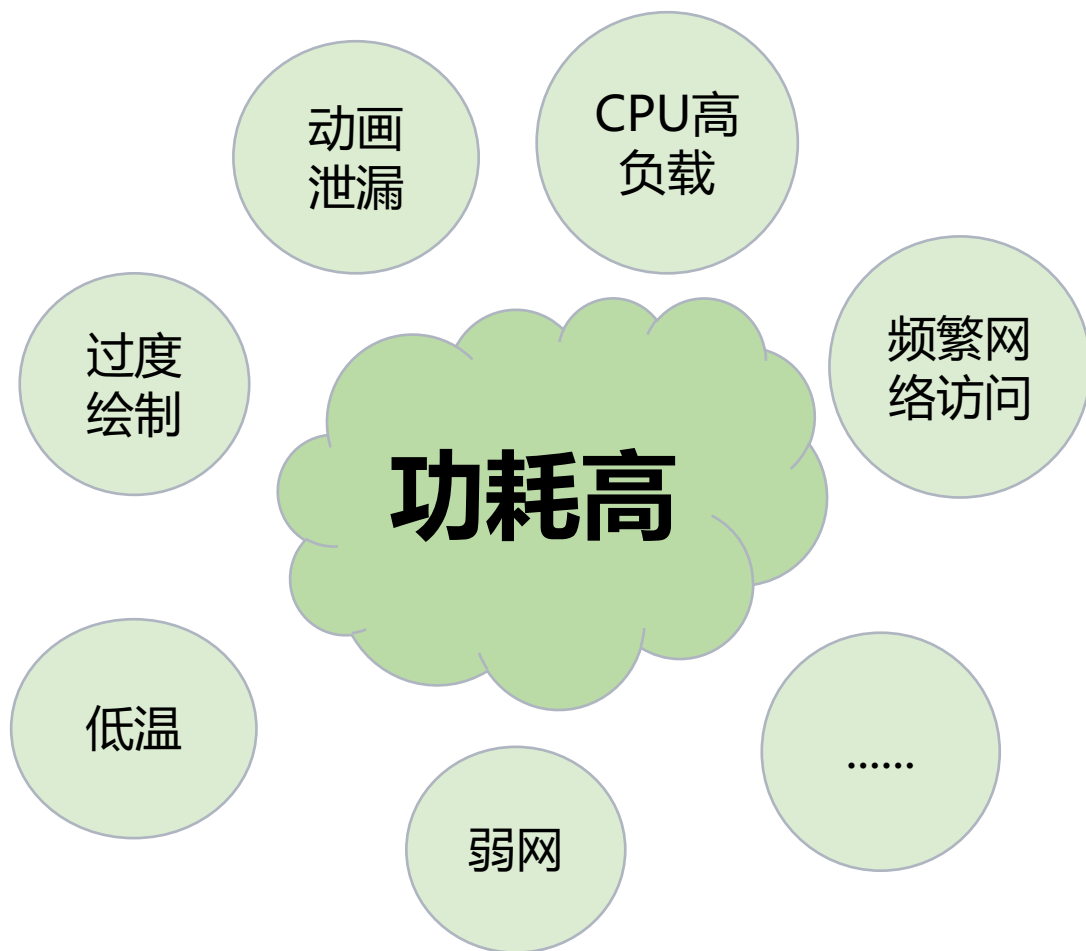
01

抖音功耗优化的开展思路

01_ 功耗优化的背景

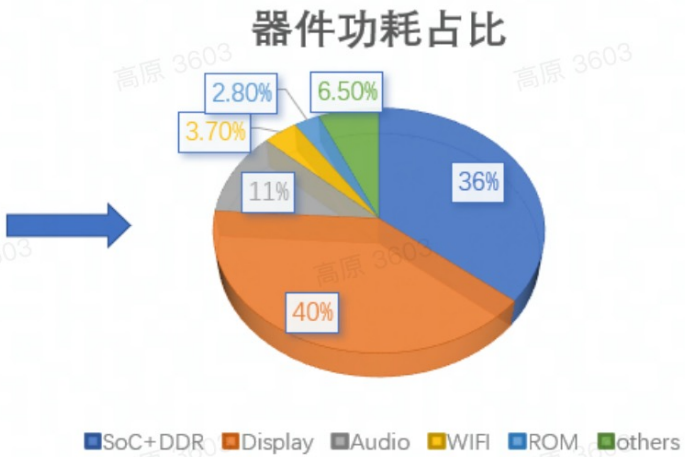


02_ 优化方案的挖掘



如何做功耗优化呢？

02_ 优化方案挖掘_器件角度



推荐页功耗拆解

功耗分析 找出哪些器件耗电

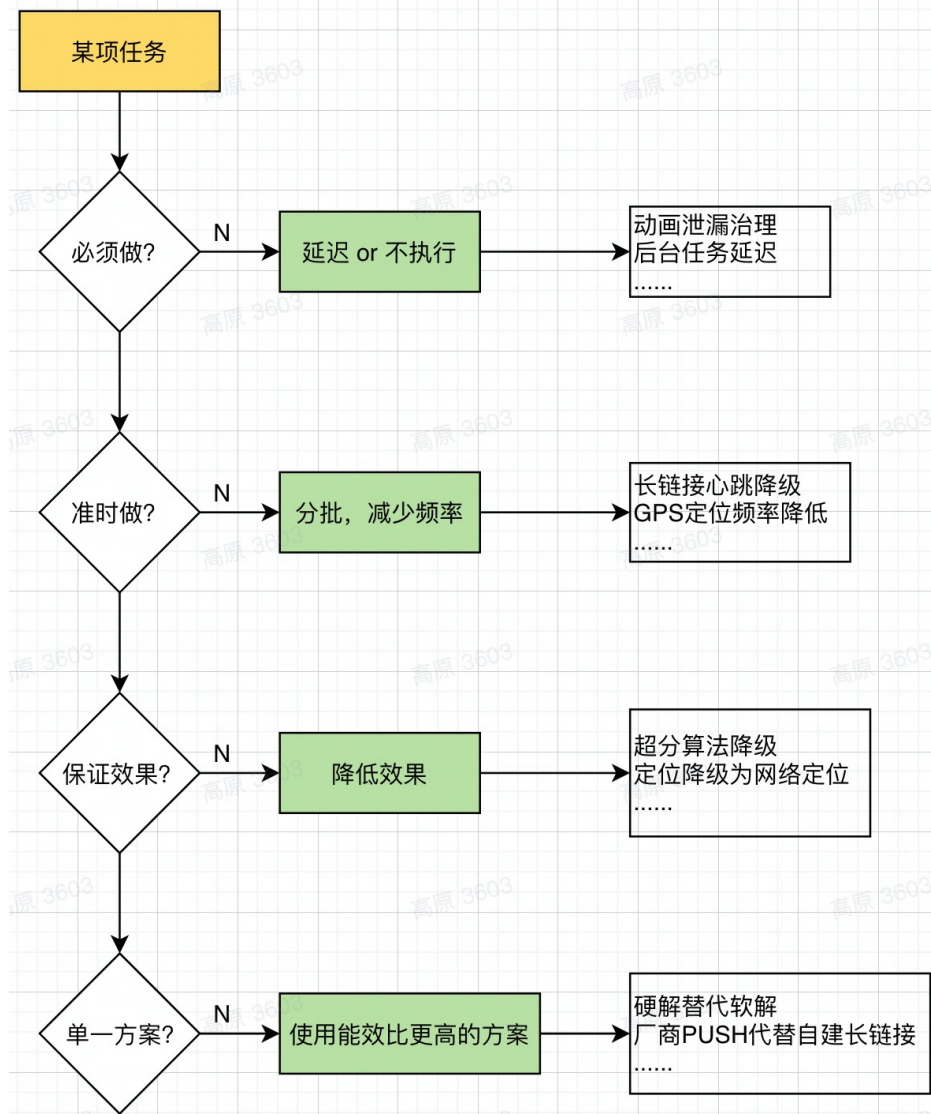
优化思路 怎么减少器件使用

器件	功耗影响因子
Display	亮度，显示内容，刷新率
CPU	工作频率，工作负载，运算内容
GPU	工作频率，工作负载，绘制内容，分辨率，绘制帧率
WIFI	信号，协议（影响收发速率），工作频率（2.4G，5G）
Audio	音量，音效算法

- 降低亮度
- 降低动画帧率
- 减少数据收发
- 降低音量
- 治理CPU高负载异常

03_优化方案挖掘_器件角度

器件	功耗影响因子
Display	亮度, 显示内容, 刷新率
CPU	工作频率, 工作负载, 运算内容
GPU	工作频率, 工作负载, 绘制内容, 分辨率, 绘制帧率
WIFI	信号, 协议 (影响收发速率), 工作频率 (2.4G, 5G)
Audio	音量, 音效算法
GPS	频率, 定位精度
Modem	天线信号强度, 协议 (如5g功耗远高于4g) 环境 (如高铁上会频繁测量搜网)
Camera	摄像头 (主摄, 微距, 广角, 前置, tof等) 工作模式 (预览, 拍照, 录像) 分辨率, 录制帧率

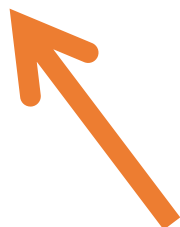


02_ 优化方案挖掘_器件角度

Camera

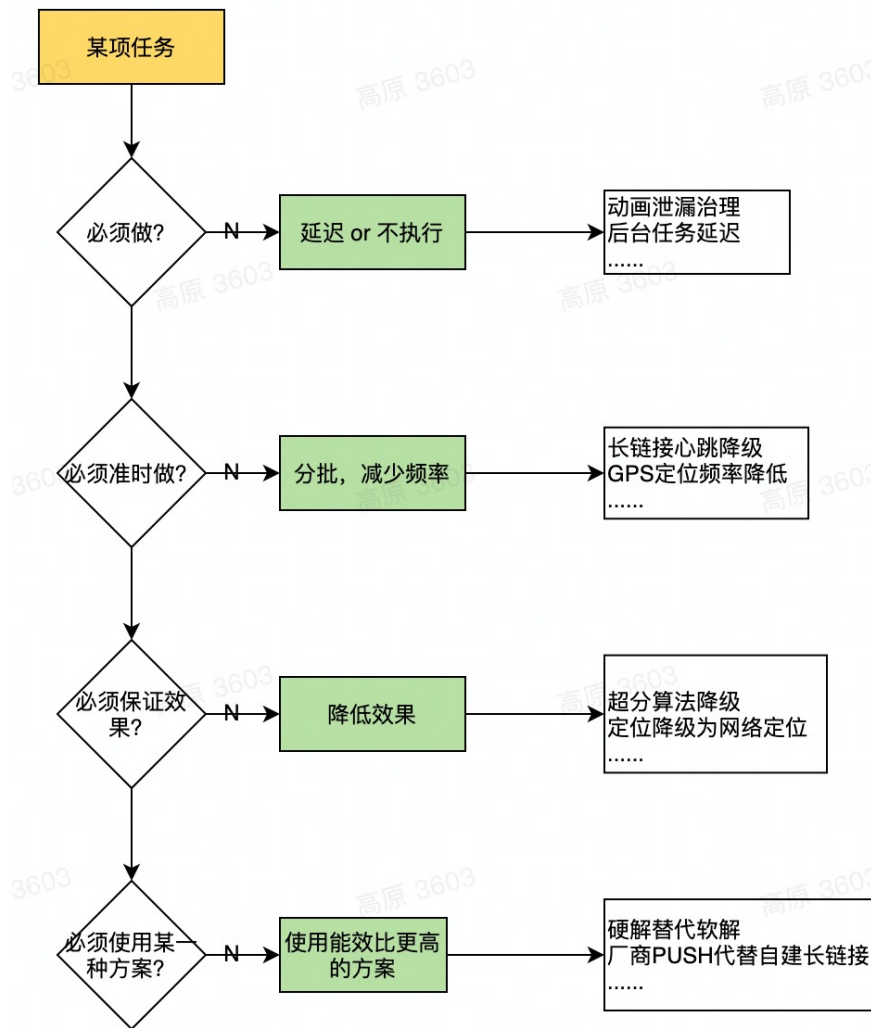
- 高分辨率高帧率的录制会带来快速的功耗消耗和温升
- 开播场景，Camera功耗 200mA+，占整机的25%以上

开播请求的采集帧率是30fps，但只使用了15fps

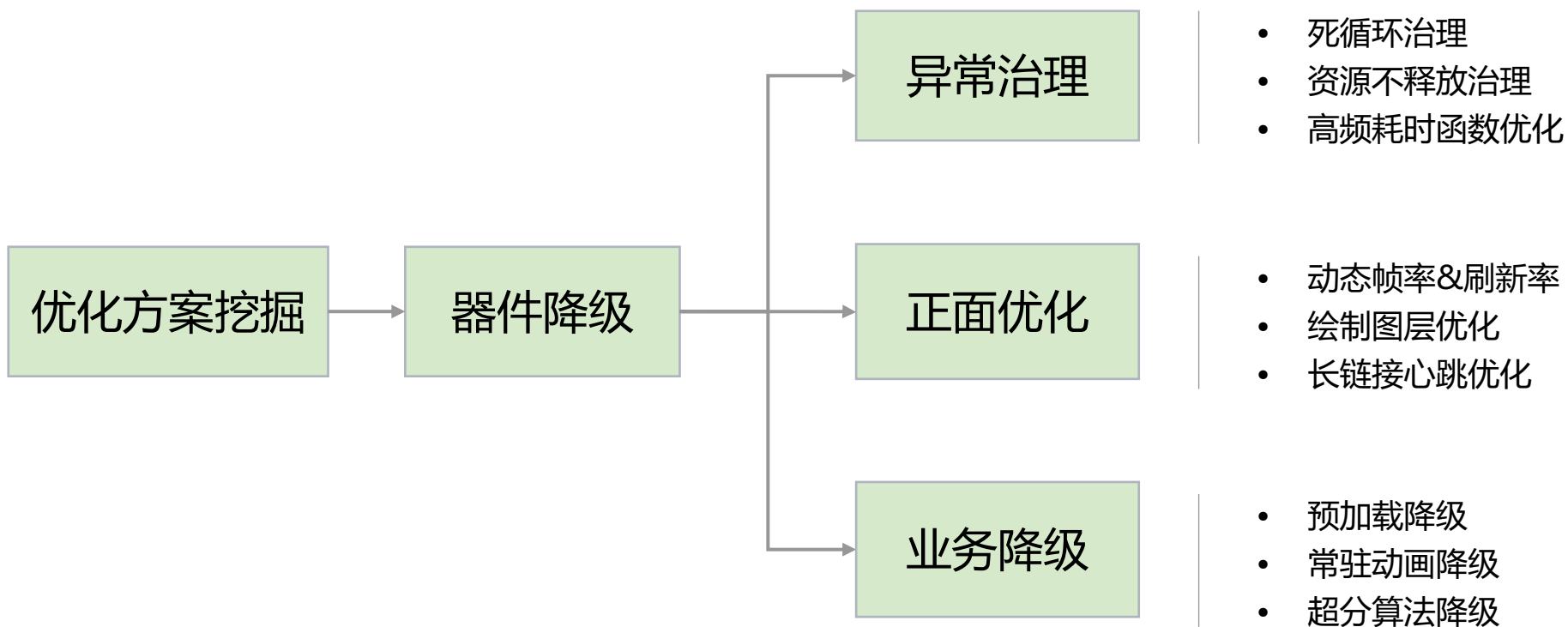


优化手段：主动下调采集帧率30FPS --> 15FPS

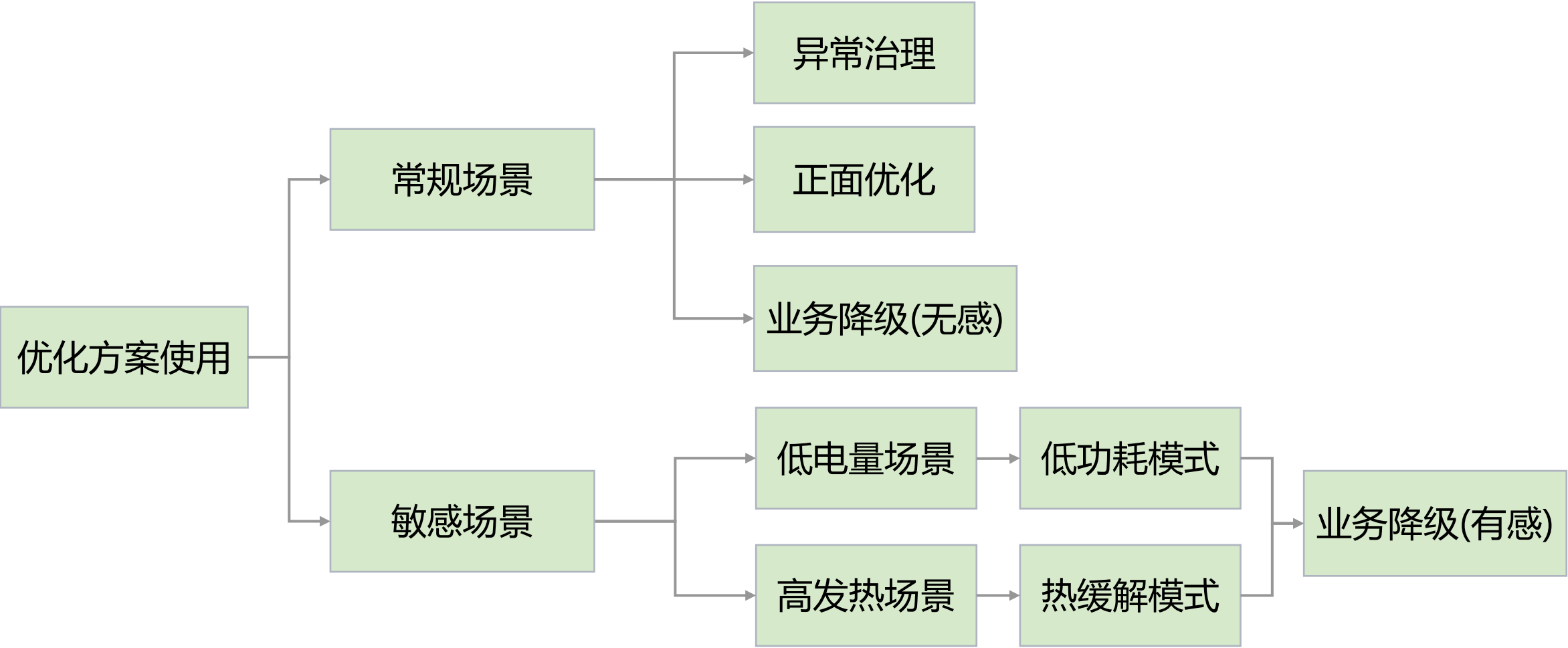
收益：CPU -13% 整机功耗 -120mA



02_ 优化方案挖掘_器件角度



03_ 优化方案使用_体验角度

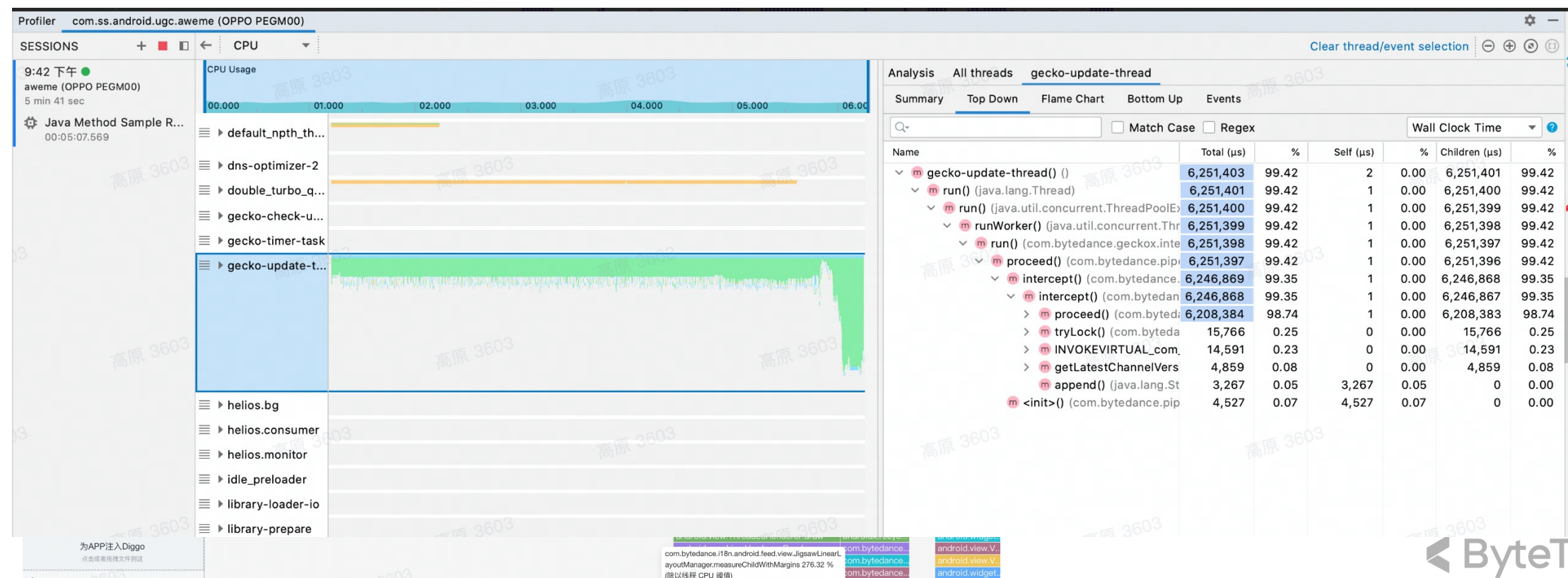


02

优化案例解析

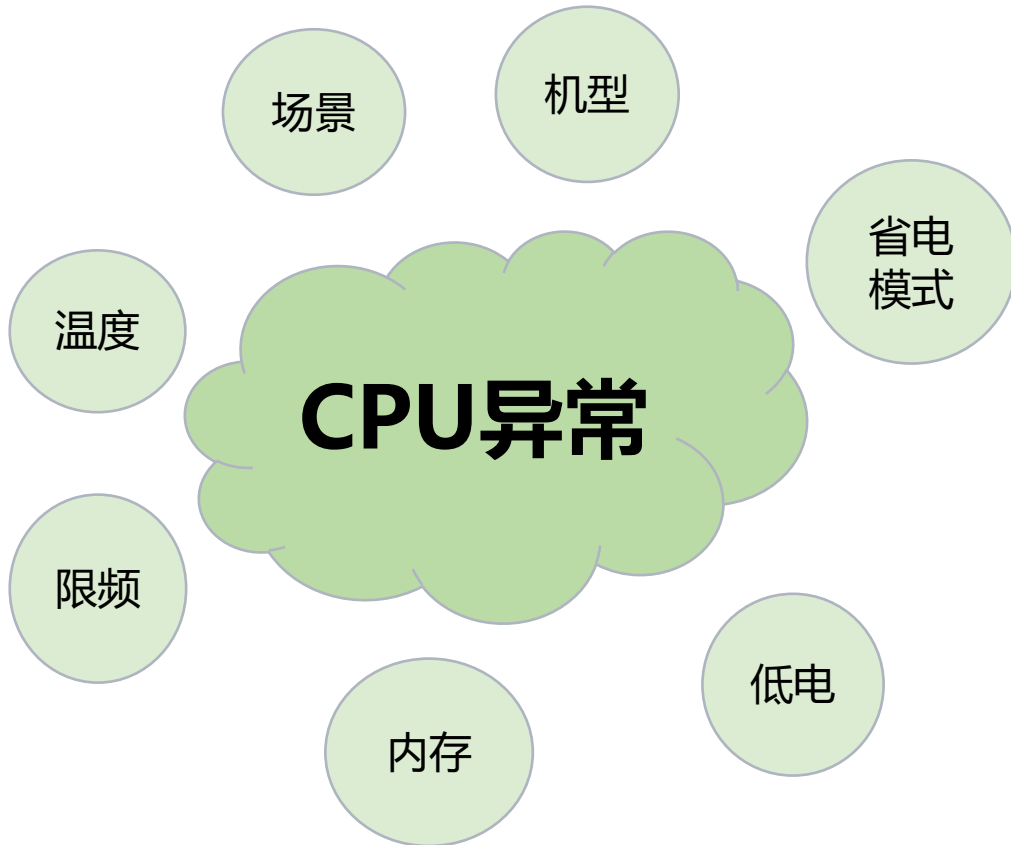
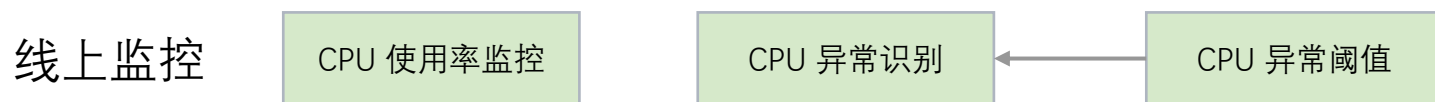
01_CPU异常治理

CPU高负载异常是最常见的一类功耗问题。引起CPU高负载的原因有很多，通常是业务开发的不规范导致的。我们主要通过线下和线上的工具，识别出CPU异常，进行进行Case by case的优化。



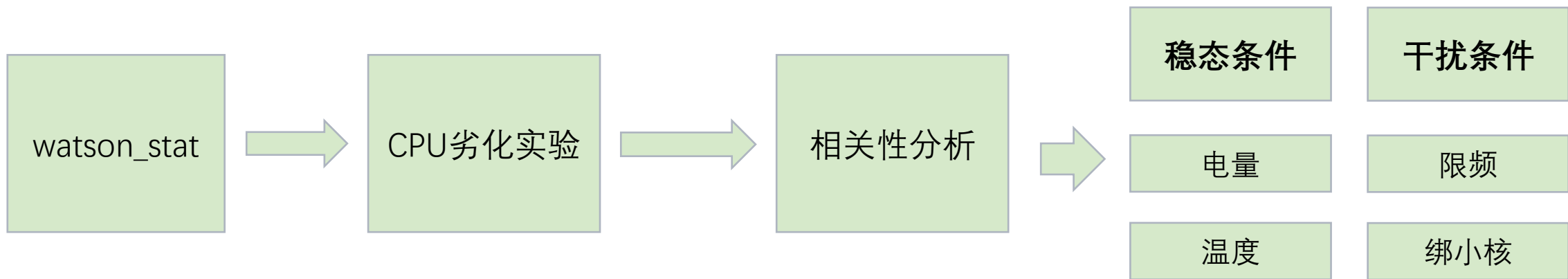
01_CPU异常治理

CPU高负载异常是最常见的一类功耗问题。引起CPU高负载的原因有很多，通常是业务开发的不规范导致的。我们主要通过线下和线上的工具，识别出CPU异常，进行Case by case的优化。



怎么确定异常阈值？

01_CPU异常治理



我们得到了下面的结论：

- 电池温度一般在37度以下，不会触发CPU温控限制
- 未充电且30%电量以上，CPU使用率比较稳定
- 省电模式对CPU影响较大
- 网络类型对CPU影响很小

CPU异常阈值：

- 电池电量 ≥ 30 ，电池温度 ≤ 37 度
- 不同CPU型号 * 不同场景
- CPU劣化组的P90使用率

01_CPU异常治理_死循环

死循环类：循环退出条件达不到，异常分支处理不当，handler消息循环等

```
// 边界条件未满足, 无法break
while (true) {
    ...
    if (shouldExit()) {
        break
    }
}

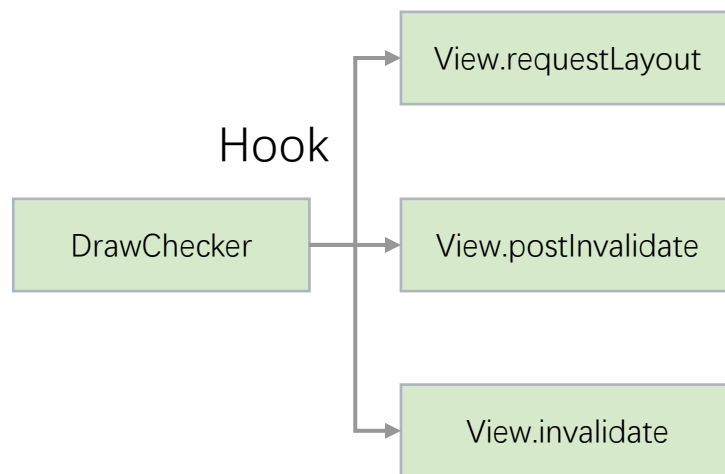
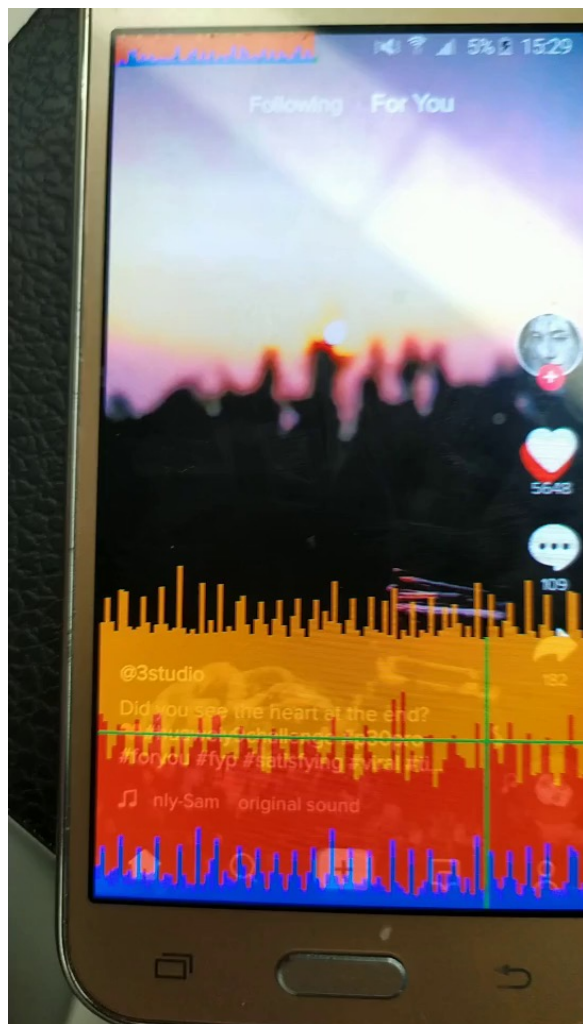
// 异常处理不妥当, 导致死循环
while (true) {
    try {
        do something;
        break;
    } catch (e) {
    }
}

// 消息处理不当, 导致Handler线程死循环
void handleMessage(Message msg) {
    //do something
    handler.sendMessage(MSG)
}
```

```
// 方法逻辑有裁剪, 仅贴出主要逻辑
private JSONArray packMiscLog() {
    do {
        .....
        try {
            cursor = mDb.query(TABLE_MISC_LOG, MISC_LOG_COLS, where, whereArgs);
            int n = cursor.getCount();
            .....
            if (start_id >= max_id) {
                break;
            }
            .....
            mDb.delete(TABLE_MISC_LOG, deleteWhere, deleteArgs);
        } catch (Exception e) {
        } finally {
            safeCloseCursor(cursor);
        }
    } while (true);
    return ret;
}
```

01_CPU异常治理_资源不释放

资源不释放：动画泄漏，音频泄漏，wakelock不释放等



performance: requestLayout泄漏, View: LinearLayout ; id: leak_view

01_CPU异常治理_资源不释放

资源不释放：动画泄漏，音频泄漏，wakelock不释放等

com.ss.android.ugc.aweme.lite	NA
com.ss.android.ugc.aweme.lite	NA
com.ss.android.ugc.aweme.lite	1041/0/3028/AudioMix/audio:
com.ss.android.ugc.aweme.lite	NA
com.ss.android.ugc.aweme.lite	10238/-1/4532/AudioMix/com.
screen_off	10238/-1/10247/AudioMix/cor
screen_off	10238/-1/10005/AudioMix/cor
screen_off	10238/-1/9999/AudioMix/com.
screen_off	10238/-1/10001/AudioMix/cor
screen_off	10238/-1/10001/AudioMix/cor
screen_off	10238/-1/9998/AudioMix/com.
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10005/AudioMix/cor
screen_off	10238/-1/9995/AudioMix/com.
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10001/AudioMix/cor
screen_off	10238/-1/9999/AudioMix/com.
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10001/AudioMix/cor
screen_off	10238/-1/9999/AudioMix/com.
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10000/AudioMix/cor
screen_off	10238/-1/10001/AudioMix/cor
screen_off	10238/-1/9999/AudioMix/com.
screen_off	10238/-1/10001/AudioMix/cor

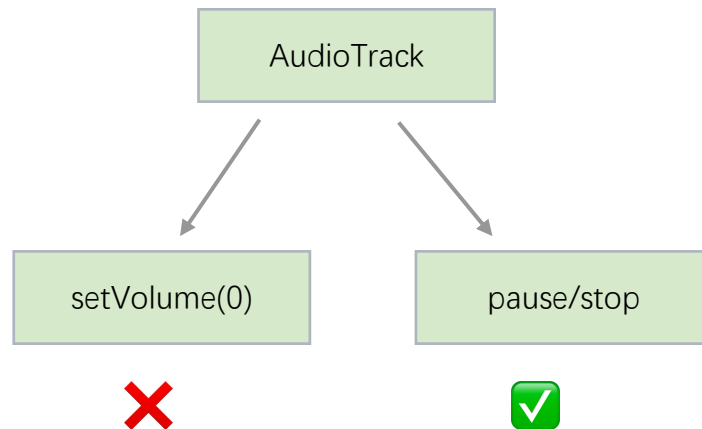
```
Wake Locks: size=4
mLock:87094937 SCREEN_BRIGHT_WAKE_LOCK 'WindowManager' ON AFTER RELEASE ACQ=-1m27s96ms (uid=1000 pid=1591)
mLock:85008087 PARTIAL_WAKE_LOCK 'AudioMix' ACQ=-1m12s874ms LONG (uid=1041 ws=WorkSource{10390})
```

```
String16 AudioFlinger::ThreadBase::getWakeLockTag()
```

```

{
    switch (mType) {
        case MIXER:
            return String16("AudioMix");
        case DIRECT:
            return String16("AudioDirectOut");
        case DUPLICATING:
            return String16("AudioDup");
        case RECORD:
            return String16("AudioIn");
        case OFFLOAD:
            return String16("AudioOffload");
        case MMAP_PLAYBACK:
            return String16("MmapPlayback");
        case MMAP_CAPTURE:
            return String16("MmapCapture");
        case SPATIALIZER:
            return String16("AudioSpatial");
        default:
            ALOG_ASSERT(false);
            return String16("AudioUnknown");
    }
}

```



01_CPU异常治理_高频调用耗时函数

高频调用耗时函数：比如md5校验，网络建链，正则表达式等

```
GMT+08:00 08:46:44.861 [26620:26620*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 08:46:45.913 [26620:26620*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 08:46:46.968 [26620:26620*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:08.215 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:08.218 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:09.069 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:09.077 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:09.274 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:09.277 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:10.124 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:10.127 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:10.315 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
GMT+08:00 09:07:10.328 [29671:29671*] [E] [iesimcore] [ , ] com.bytedance.frameworks.baselib.network.http.exception.NetworkNotAvailabeException: network not available
```

// PowerManager提供的两种判断当前是否处于Doze模式方法

```
public boolean isDeviceIdleMode() {
    try {
        return mService.isDeviceIdleMode();
    } catch (RemoteException e) {
        throw e.rethrowFromSystemServer();
    }
}
```

```
@SdkConstant(SdkConstant.SdkConstantType.BROADCAST_INTENT_ACTION)
public static final String ACTION_DEVICE_IDLE_MODE_CHANGED
    = "android.os.action.DEVICE_IDLE_MODE_CHANGED";
```

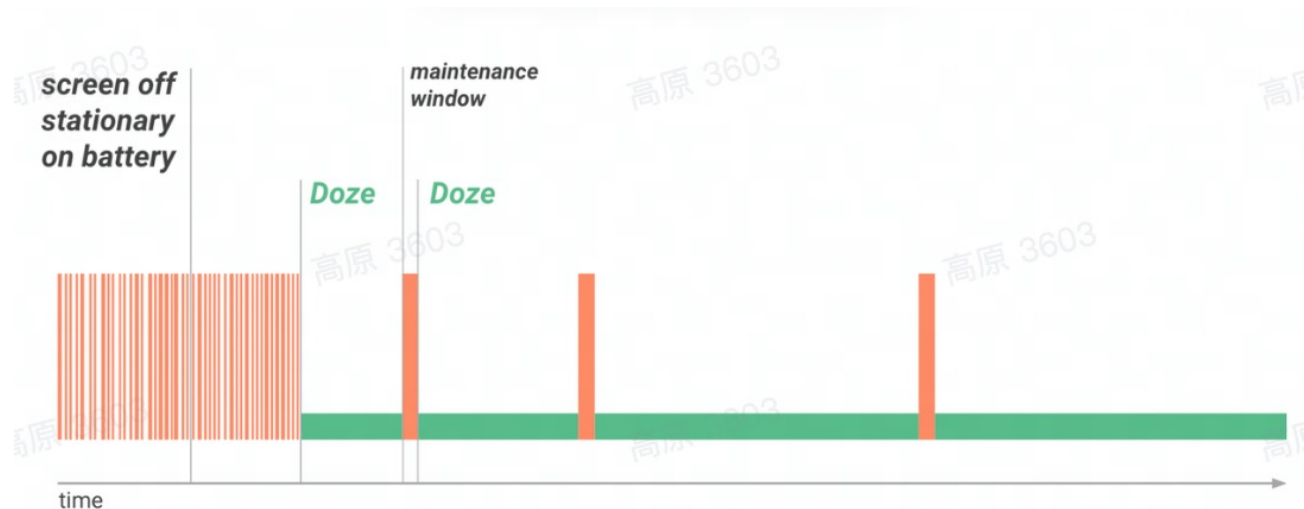
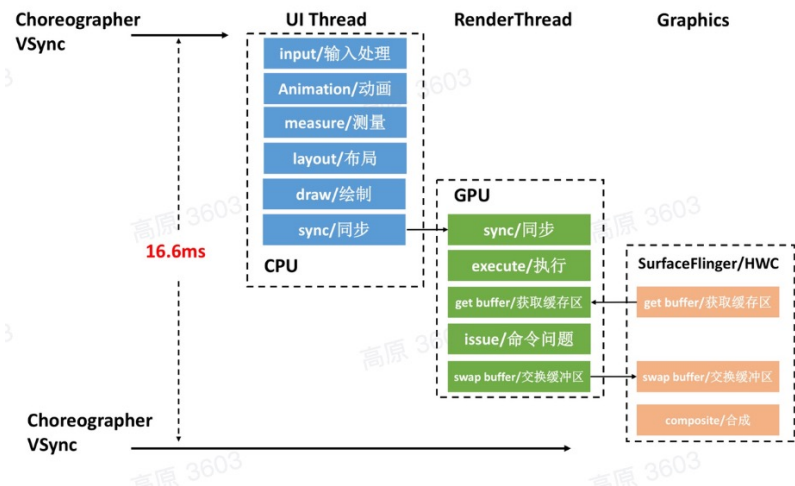


图 1. 低功耗模式提供了周期性维护期，让应用使用网络并处理待处理的活动。

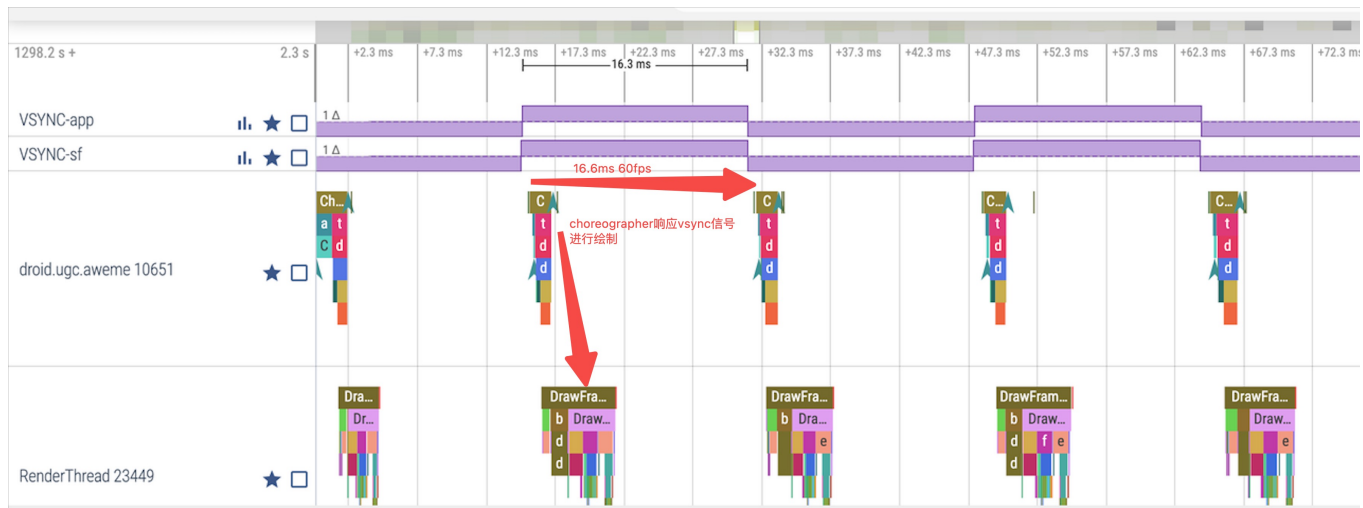
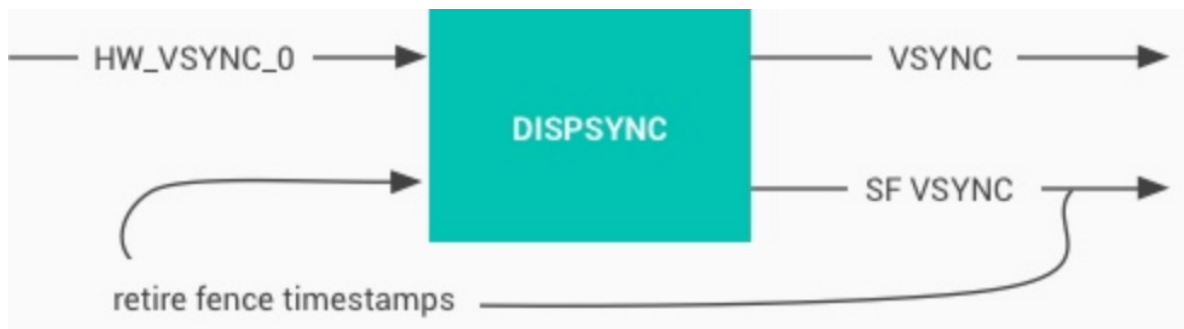
02_动态帧率

背景：绘制帧率可以很明显的影影响GPU功耗。抖音大部分的视频帧率都是30fps，叠加上各种动画(音乐转盘，跑马灯，进度条，活动动画)，绘制帧率会处于60fps，降低绘制帧率可以有效优化GPU耗电。



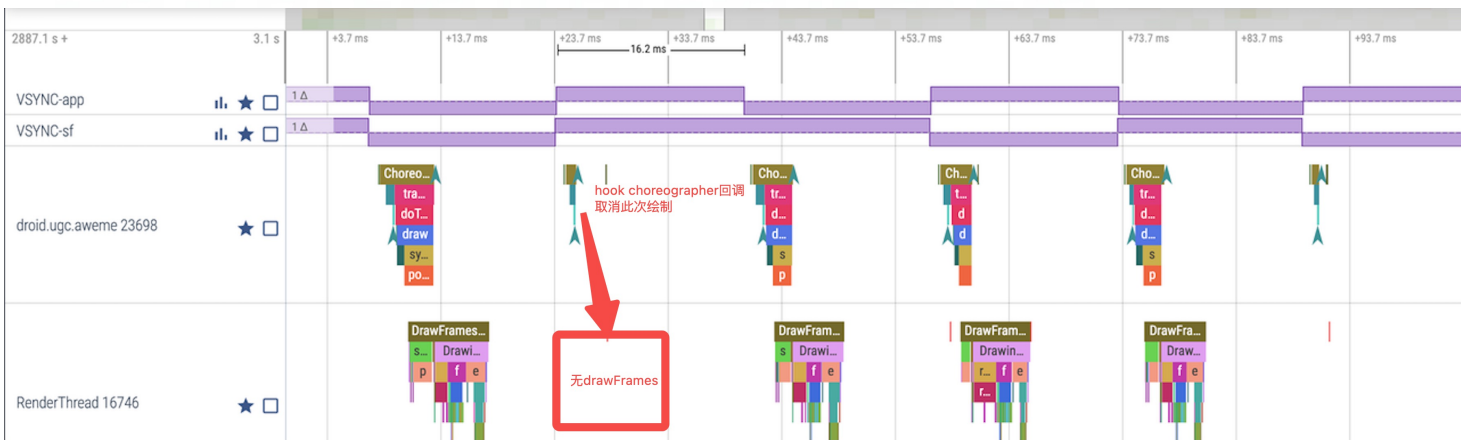
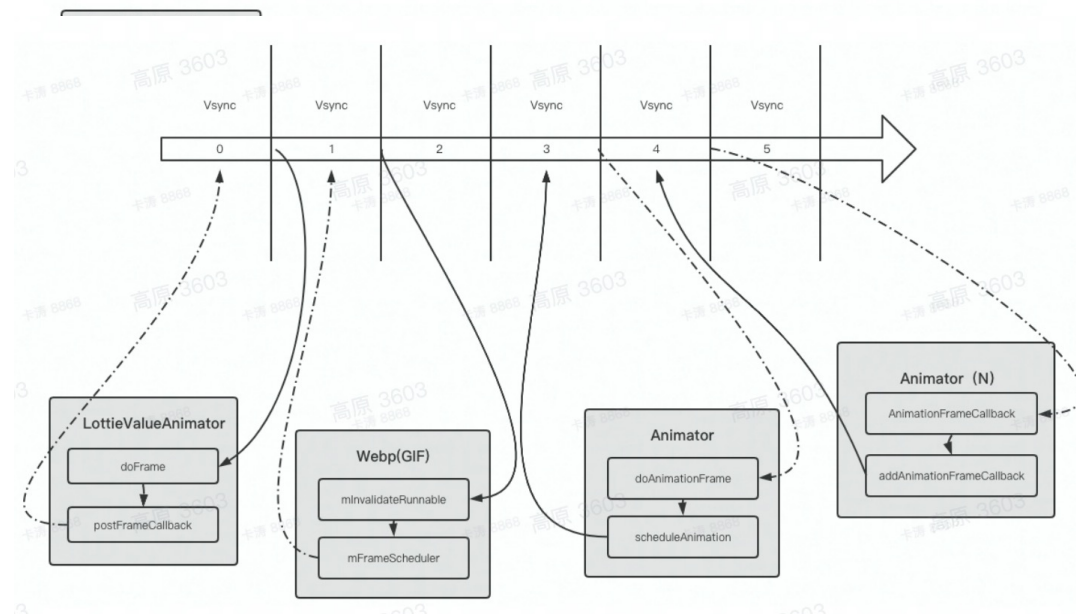
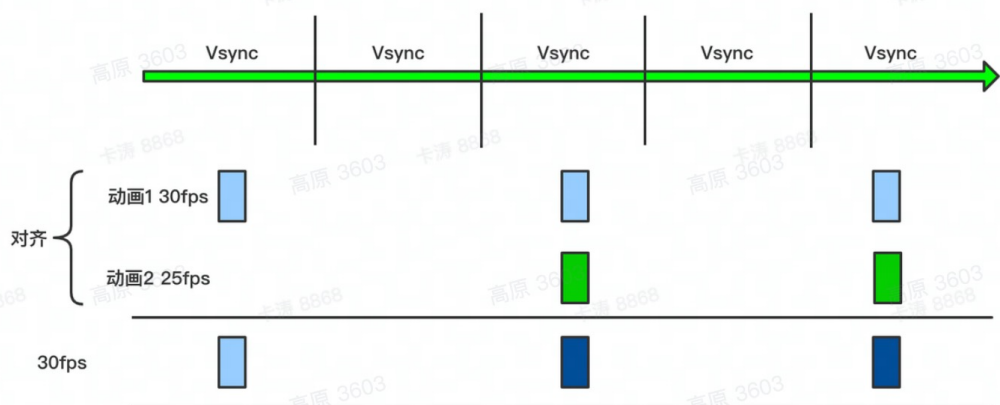
优化方向：

- 自身帧率优化
 - 动画帧率控制
 - 动画帧率对齐
 - 动画与播放器帧率对齐
- 厂商合作
 - 调节vsync回调频率，实现动态帧率



02_动态帧率_Kita

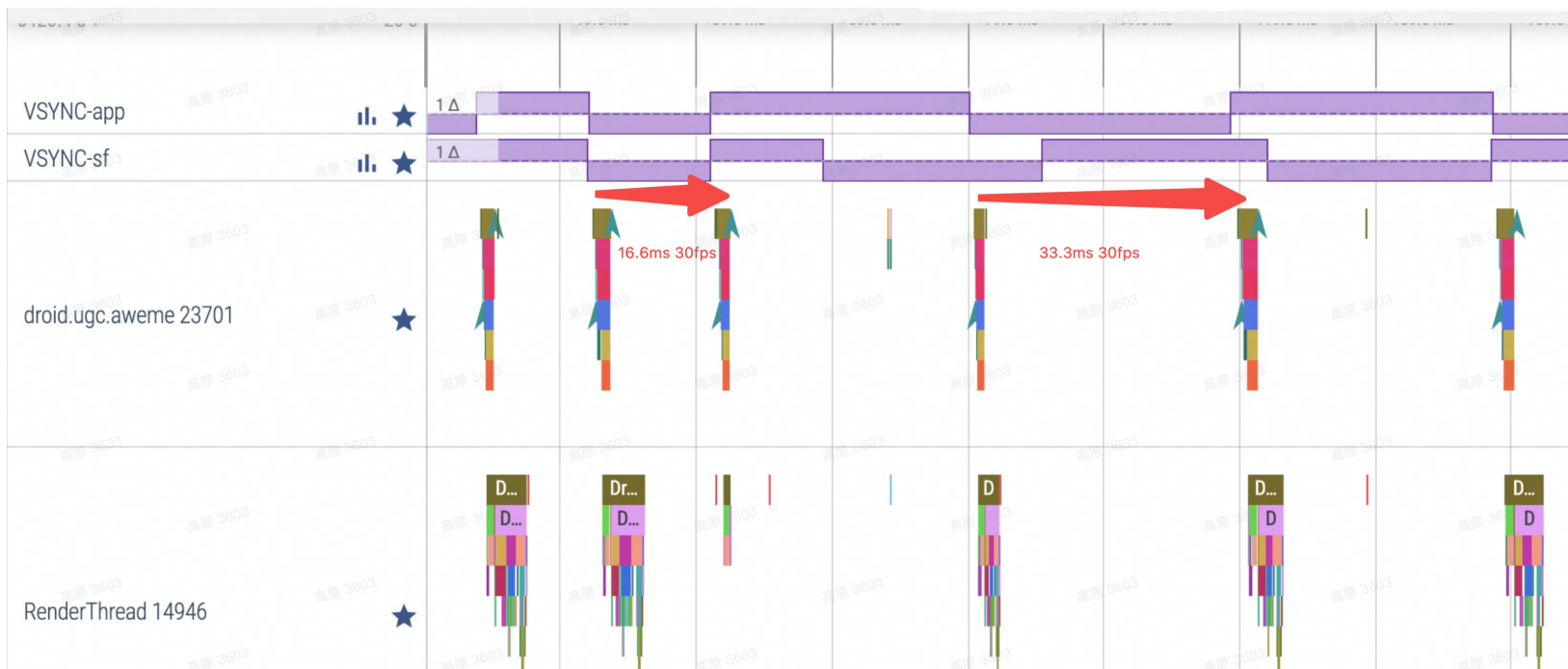
Kita框架通过托管多种动画的绘制流程，将整体的动画调度逻辑在kita controller中进行统一管理，通过降帧，对齐后，实现了整体绘制帧率的降低



场景	60帧 (mA)	30帧 (mA)	收益 (mA)
主视频场景	305.5	273.7	31.8
双列视频列表	319.7	278.5	41.2
详情视频页面	320.5	276.2	44.3

02_动态帧率_厂商合作

厂商通过调节Vsync的回调频率或者利用LTPO屏幕的动态刷新率能力，都可以从系统侧实现对应用绘制帧率的控制，我们和厂商合作，在视频推荐页通知系统降低帧率到30fps，在部分场景恢复正常帧率(如弹幕，高帧率视频场景)，可以在保证用户体验的情况下获取更大的功耗收益。



光子引擎

MIUI 14 不仅流畅
也更加省电

刷微博电量节省

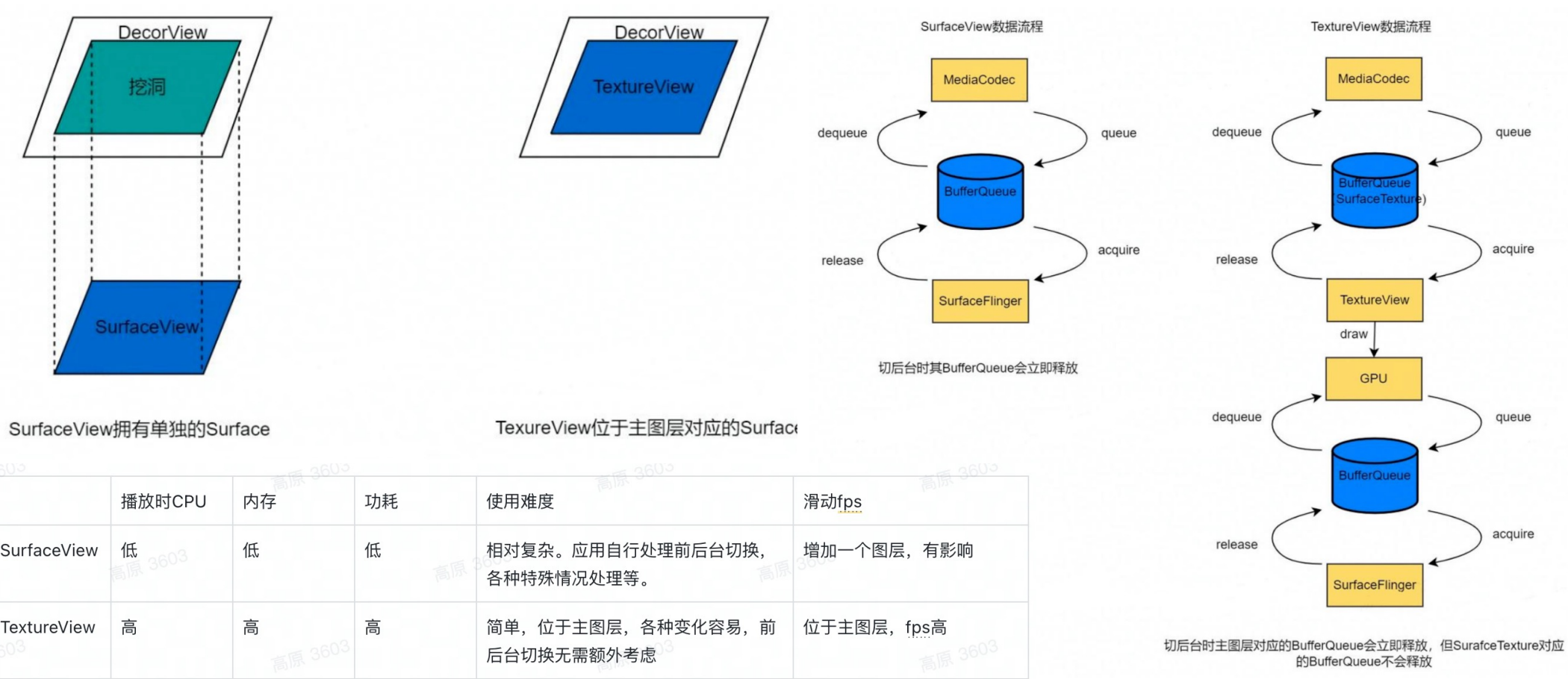
16%↓

刷抖音电量节省

22%↓

03_SurfaceView替换TextureView

背景：TextureView和SurfaceView是两个最常用的播放视频控件。TextureView控件位于主图层上，解码器将视频帧传递到TextureView对象还需要GPU做一次绘制才能在屏幕上显示，所以其功耗更高，消耗内存更大，CPU占用率也更高。



收益：CPU -13% 整机功耗 -20mA

04_低功耗模式

背景：低电量情况下，用户对功耗更加敏感，我们需要更激进的功耗优化措施

- 未充电的情况下，剩余电量 $\leq 5\%$ ，单次停留时长开始显著下降
- 剩余最低电量在 $2\%-5\%$ 的，会话占比 1.2% ，时长在15-18分钟左右，对比剩余10%的平均时长27分钟，要损失约11分钟

低功耗模式

- 动画暂停
- 超分算法降级
- 定位服务降级
- 播放器降级
- 关闭离屏渲染
- 视频码率降级
- Sensor降级

生效场景

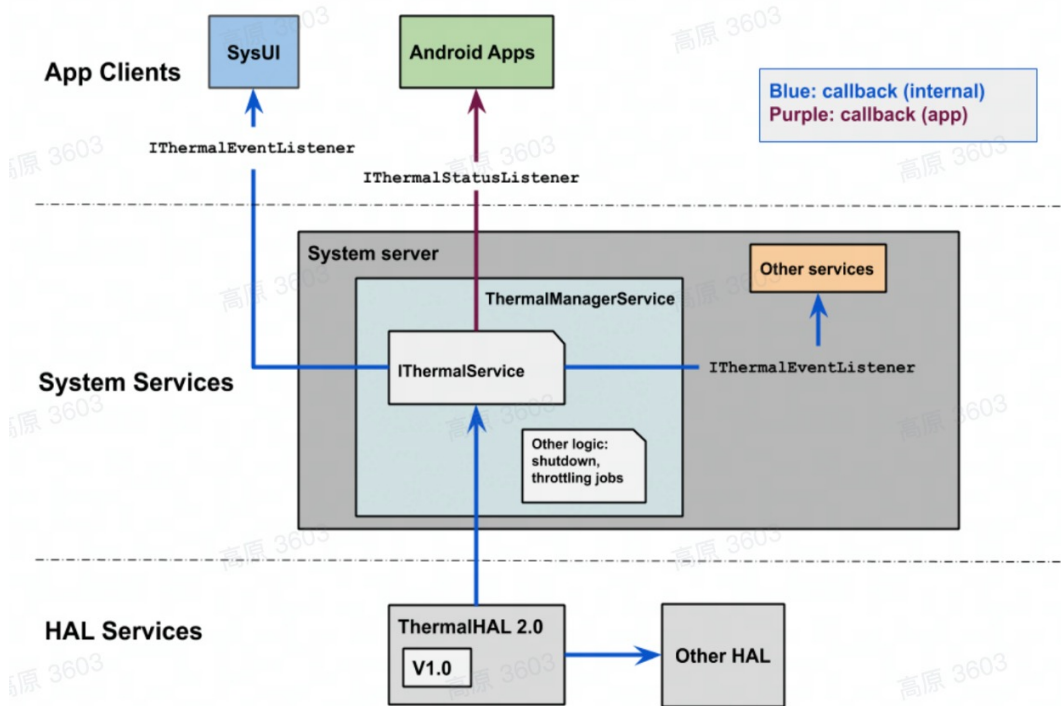
- 非充电情况下电量低于30%
- 易发热机型增加开启时长

功耗收益：20mA

05_热缓解

背景：手机发热会影响用户体验，并且系统会通过限制器件的使用，减少发热量。前台应用不及时进行调整的话很可能会出现卡顿。

优化方案：监控手机发热，在发热前主动进行激进的降级手段，减少资源消耗，减少发热以及卡顿的发生



Android10 热缓解框架

热状态代码	说明和建议用法
NONE (0x0)	未限制。 此状态代码可用于实施保护操作，例如，检测从 NONE (0x0) 到 LIGHT (0x1) 的时间段（从 t0 到 t1）的起始时间。
LIGHT (0x1)	轻微限制。用户体验不受影响。 在此阶段，请对设备采取温和的缓解操作。例如，不要提频或采用低效频率（仅适用于大核心）。
MODERATE (0x2)	中等限制。用户体验没有受到很大影响。 执行的热缓解操作会影响前台 Activity，因此应用应立即减少耗电量。
SEVERE (0x3)	严重限制。用户体验在很大程度上受到影响。 在此阶段，设备热缓解措施应该会限制系统容量。这可能会导致显示卡顿和音频抖动等负面影响。
CRITICAL (0x4)	平台已采取一切措施减少耗电量。 设备热缓解软件已经以最低容量运行所有组件。
EMERGENCY (0x5)	平台中的关键组件因热状况而即将关闭。 设备功能受限。这是系统在设备关机前发出的最后一次警告。在此阶段，调制解调器和移动数据网络等一些功能会完全关闭。
SHUTDOWN (0x6)	立即关机。 鉴于此阶段的严重级别，应用可能无法收到此通知。

官方热缓解框架需要各OEM厂商适配，但OEM并没有适配此框架，而是通过开放SDK里提供了类似的能力，支持度和灵敏度，兼容性更优于谷歌热缓解框架

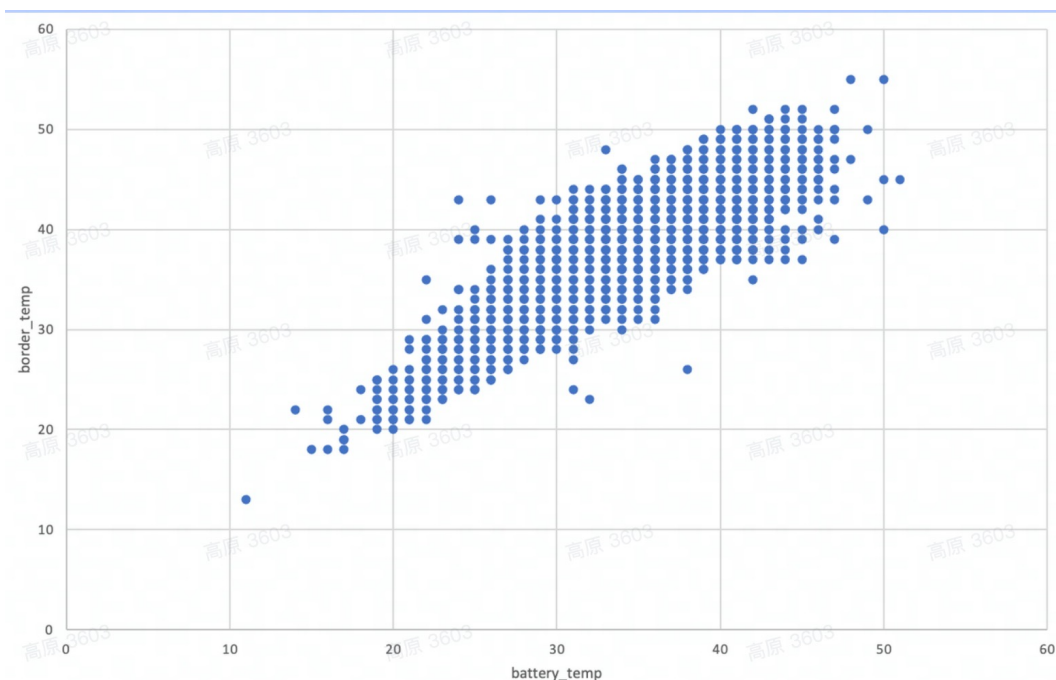
06_热缓解

Unitm：对各厂商的SDK进行适配，提供统一的温控级别



优点：提供壳温，高热识别更灵敏

缺点：老机型不支持，需要升级版本，覆盖率不高



电池温度VS壳温相关性分析

- 系统在壳温平均43度左右会开启温控限制，39度左右解除限制
- 壳温与电池温度相关，壳温高于电池温度约2度

03

总结&展望

- 总结
 - 从器件角度出发寻找功耗优化手段
 - 从体验角度出发，衡量如何使用降级手段(低功耗模式，热缓解)
- 展望
 - 针对常见功耗问题，建立完善的功耗异常检测框架
 - 完善功耗防劣化能力

更多详细内容：<https://mp.weixin.qq.com/s/Sj3hSYpSoGIuNIUpu7ZQeg>



THANKS.